

Claude Code x Stata

Building a Domain-Specific AI Research Agent

Stata Module | UEBS Statistics Study Group
27 March 2026 | Runzhi Tian & Yifan Qi



Agenda

Six steps from a generic chatbot to a fully autonomous Stata research agent

0. WSL Environment

Windows only - Linux subsystem

1. Install & PATH Config

Claude Code + Stata path mapping

2. Safe Workspace

Project directory isolation

3. Stata Skill Injection

Econometrics knowledge base (37 docs)

4. MCP Closed Loop

Write > Run > Read > Fix automatically

5. CLAUDE.md Memory

Persistent research preferences

Why Stata Needs Special Configuration

Path Invisible

Stata is not on the system PATH by default - Claude Code cannot discover or call it

Proprietary Docs

Official manuals exist as large PDFs - feeding them raw burns tokens fast

Unique Paradigm

do-files, macros, globals, preserve/restore - needs domain-specific context

The Configuration Stack

WSL / PATH



Stata Skill



PDF Tools



MCP Server



CLAUDE.md

Step 0: WSL Environment Setup

Windows users only - Mac/Linux skip to Step 1

1. Install WSL (PowerShell as Admin):

***Search PowerShell at Start and open it as administrator**

```
wsl --install  
  
wsl.exe --list --online  
  
wsl.exe --install Ubuntu
```



Windows PowerShell

It will then download and install the subsystem of Linux (Ubuntu)

2. Restart > Set username & password > Update:

```
Create a default Unix user account: xxxxx  
New password: (quietly record what you typed and won't be displayed)  
Retype new password: (quietly record what you typed and won't be displayed)  
sudo apt update && sudo apt upgrade -y
```



Common Gotchas

1. Password is invisible when typing - that is normal, just type blind and press Enter
2. Ctrl+V does not work in terminal - use Right Click or Ctrl+Shift+V to paste (y)
*In PowerShell, right-click the title bar, open *Properties*, and enable **"Use Ctrl+Shift+C/V as Copy/Paste."**
3. If paste still fails: Right-click title bar > Properties
> enable QuickEdit Mode

Sudo: superuser do
(administrative authorization)

Step 1: Install Claude Code & Map Stata Path

Install Node.js & Claude Code

```
sudo apt install nodejs npm -y
```

Run the command with administrative privileges to install Node.js and npm using apt, automatically confirming all prompts.

```
sudo npm install -g @anthropic-ai/claude-code
```

to globally install the **@anthropic-ai/claude-code** package using npm.

Without sudo > EACCES: permission denied. Always use sudo.

Map Stata Path (WSL, Stata 18 SE)

```
Nano ~/.bashrc
```

```
alias stata="/mnt/c/Program\ Files/Stata18/StataSE-64.exe"
```

```
'Ctrl+O' and 'Enter' to save; 'Ctrl+X' to exit to bash
```

```
source ~/.bashrc
```

```
tata
```

First Launch

```
$ claude
```

```
Welcome to Claude Code v2.x
```

```
Choose text style:
```

```
> 1. Dark mode
```

```
2. Light mode
```

```
> Browser opens for auth
```



Verify (type code below)

```
$ stata
```

```
# Stata window pops up
```

```
$ stata & # run in background
```

Step 1 (continue): Map Stata Path

```
# Add an "alert" alias for long running commands. Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ] && echo terminal || echo error}" "$(history|tail -n1|sed -e '\''>
# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi
alias stata="/mnt/c/Program\ Files/Stata18/StataMP-64.exe"
```

Press '↓', go down to the bottom

← Type the code and 'Ctrl+O + Enter' to save

```
~$ source ~/.bashrc
~$ stata
```

'Ctrl+X' go back to bash and type ← code line by line, stata will then pop up

Step 2: Create a Safe Workspace

DANGER: Never launch here

```
/mnt/c/WINDOWS/system32
```

AI has read/write/execute access here.

One wrong command = ****system crash****.

SAFE: Your Home Directory

```
mkdir ~/create a new folder name, e.g.,  
my_stata_project or  
cd /home/username/my_stata_project or  
cd ~/my_stata_project  
claude  
: /my_stata_project$ claude
```

Optional: `mkdir /mnt/d/stata_research`

What You Will See

Accessing workspace:

```
/home/user/my_stata_project
```

Quick safety check:

Is this a project you trust?

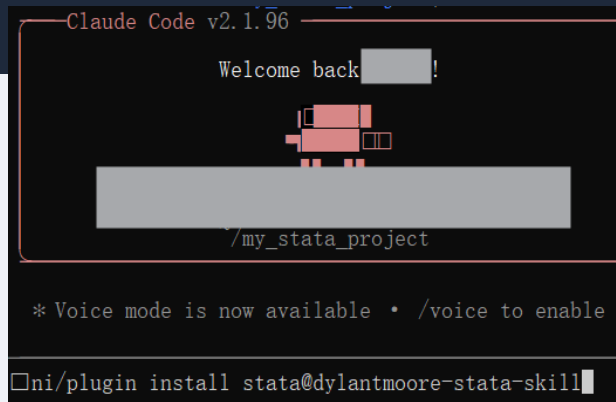
- > 1. Yes, I trust this folder
- 2. No, exit

Step 3: Inject Stata Expert Knowledge

Install the community Stata Skill - 37 reference docs covering econometrics & causal inference

Install Commands (inside Claude Code):

```
ni/plugin install stata@dylantmoore-stata-skill
```



```
> Install for you (user scope)  << Choose
  Install for all collaborators (project)
  Install for you, in this repo only (local)
```

Coverage

Data Ops

Import, clean, string, date

Estimation

OLS, panel, TS, MLE, GMM, IV

Causal

DID, RDD, PSM, treatment fx

Advanced

Survival, SEM, spatial, Lasso

Programming

do-file, macros, loops, Mata

Output

Graphs, pub-quality tables

20 Community Packages Built In

reghdfe csddid rdrobust ivreg2 estout
coefplot psmatch2 synth gtools ...

Step 4: MCP Closed-Loop Execution

Write > Run > Read output > Auto-fix - no manual intervention

Install Stata MCP Server (project scope):

```
! sudo chown -R username:username /home/username/my_stata_project/  
claude mcp add stata-mcp \  
  --env STATA_MCP_CWD=$(pwd) \  
  --scope project \  
  -- uvx --directory $(pwd) stata-mcp
```

```
claude mcp add stata-mcp \  
  --env STATA_MCP_CWD=$(pwd) \  
  --scope project \  
  -- uvx --directory $(pwd) stata-mcp
```

```
• Bash(claude mcp add stata-mcp --env STATA_MCP_CWD=$(pwd) --scope project --  
  uvx --directory $(pwd) stata-mcp)  
  └─ Running...
```

Bash command

```
claude mcp add stata-mcp --env STATA_MCP_CWD=$(pwd) --scope project --  
uvx --directory $(pwd) stata-mcp  
Add stata-mcp MCP server with project scope
```

Contains command_substitution

Do you want to proceed?

- > 1. Yes
- 2. No

1. WRITE

Generates
do-file

2. EXECUTE

stata -b do
filename.do

3. READ

Parses .log
for results

4. FIX/REPORT

Auto-debug or
report findings

🔄 If error detected in Step 3, loops back to Step 1 automatically

Global alternative: `claude mcp add stata-mcp --scope user -- uvx stata-mcp`

Step 5: Persistent Memory via CLAUDE.md

AI reads this file at every conversation start - your personal research rulebook

Initialise

```
/init
```

```
Do you want to create CLAUDE.md?
```

1. Yes
- > 2. Yes, allow all edits << Choose
3. No

Personalise (send this prompt):

```
Update CLAUDE.md:
```

1. do-file header: version 18
2. Run cmd: stata -b do <file>.do
3. Export tables with estout
4. Fixed effects via reghdfe
5. Save plots as .png in plots/

CLAUDE.md

```
# Project
```

```
## Data Paths
```

- Raw: /data/raw/
- Clean: /data/clean/
- Output: /output/

```
## Coding Rules
```

- version 18
- reghdfe for FE
- estout for tables

```
## Workflow
```

- 01_clean_*.do
- 02_analysis_*.do

Putting It All Together: Live Test

Test Prompt:

```
"Write a Stata do-file: load the built-in auto dataset, regress price on mpg and weight with robust standard errors, and report the results."
```

Agent Response Sequence:

1	<code>Skill(stata)</code>	Successfully loaded skill
2	<code>Write(regression_auto.do)</code>	Wrote 15 lines to file
3	<code>Bash(stata -b do ...)</code>	Executed in background
4	<code>Read(regression_auto.log)</code>	Parsed output results
5	Report	Coefficients + interpretation

If You See 'Interrupted'

Cause: Accidental keypress
(Esc, Ctrl+C, arrow keys)
during code generation.

Fix: Reply with:
"Continue - finish writing,
run it, and report results."

If stuck: Ctrl+C then
re-send your prompt.

From Chatbot to Research Agent

1

Hands & Feet

Direct access to local filesystem and Stata executable

2

Expert Brain

37-doc econometrics knowledge base with 20 community packages

3

Autonomous Loop

Write > Run > Debug > Report - zero manual intervention

4

Persistent Memory

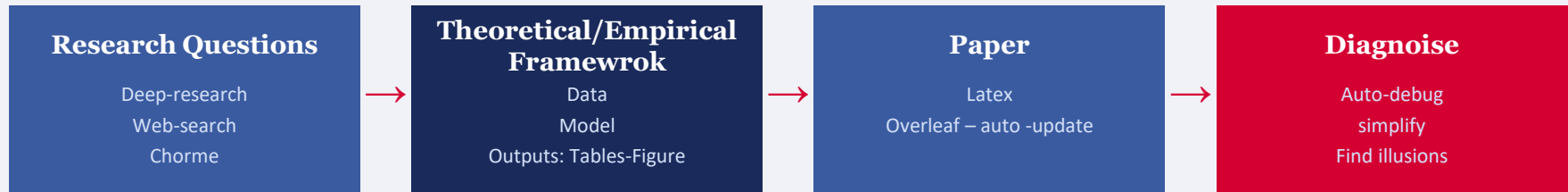
CLAUDE.md remembers your coding standards across sessions

Useful Claude Code Work Flow

Balance: Token using and speed. Always used the best model (now Opus 4.6 - thinking), but can adjust effort.

Core Files – How to use token more efficiency

```
-Readme.md
-Claude.md
-config.py
.claude - agent - memory
commands - local skills
```



You can further design four separate agents and sub-agents.

Reference: <https://github.com/ykdojo/claude-code-tips>; <https://code.claude.com/docs/en/best-practices>;

Tips:

Plan mode

Plan mode for round discussion ; and other modes for short revisions.
Comments on the plan for further discussion

Powerful slash : / + Command

1	/doctor	Diagnose and verify your Claude Code installation and settings
2	/skills	List skills
3	/Agents	Generate and View Agents
4	/tasks	Manage background tasks
5	/insights	Most useful
6	/remote-control	For Time Management (Relax)

Generate new slash/skills

Just by ask claude code:

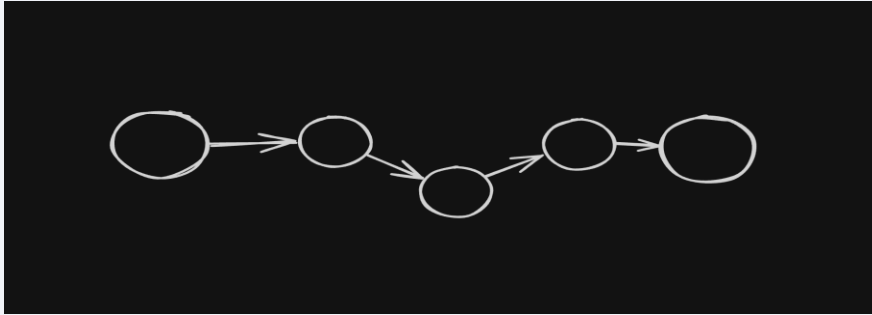
Please read the output from /insights to give me some
recommans slash and skills. (Plan mode!)

For any questions further;
Always find more specific skills from your current
project, rather than general skills-hub

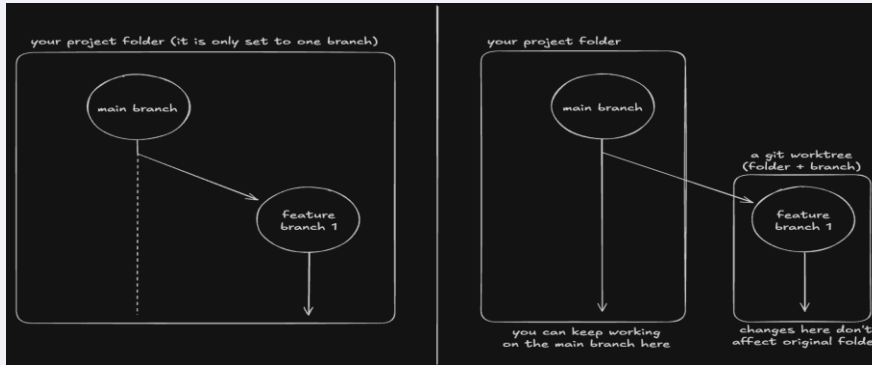
! Global-level vs Project-level

Keep Going – From Code/ Research to Vibe

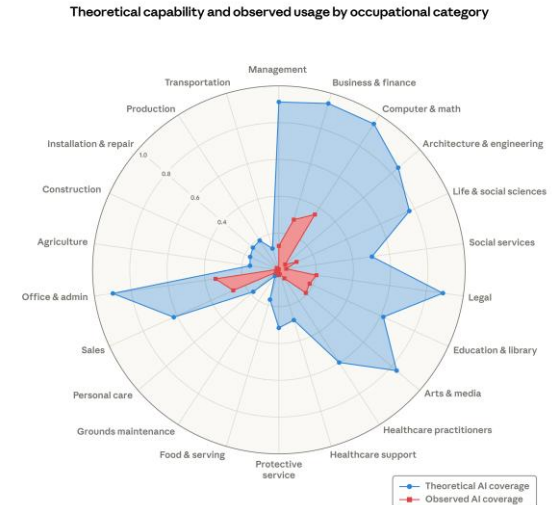
Break down large problems into smaller ones – Always start a new context window



Set Git worktrees for parallel branch work



Future co-work with AI



Keep learning!

Ask Claude Code itself or Check the /release notes

References – Update

Tips:

<https://github.com/ykdojo/claude-code-tips>

<https://github.com/shanraishan/claude-code-best-practice?tab=readme-ov-file>

Workflow

<https://psantanna.com/claude-code-my-workflow/>

<https://socialcatalystlab.org/>

Token Resource

<https://github.com/education> Github copilot can be used in the VS code with OpenAI(ChatGPT) and Anthropic (Claude) models.

Report

<https://www.anthropic.com/research/labor-market-impacts>

<https://github.com/SocialCatalystLab/ape-papers>

<https://causalinf.substack.com/p/claude-code-35-do-ai-agents-writing>